

QoE-Aware DRL for Slice Admission Control: A Live Open RAN Testbed Validation

Manoj Prasad Kunasegran

School of Engineering

Taylor's University

Subang Jaya, Malaysia

orcid.org/0009-0005-3169-1873

Wai Leong Pang*

School of Engineering

Taylor's University

Subang Jaya, Malaysia

orcid.org/0000-0001-8407-5648

Corresponding author

Swee King Phang

School of Engineering

Taylor's University

Subang Jaya, Malaysia

orcid.org/0000-0002-7877-8766

Abstract—Building on our previous work on Deep Reinforcement Learning (DRL)-driven Slice Admission Control (SAC), validated entirely in simulation, this paper moves the same control problem onto a real Open RAN testbed. We deploy a Quality of Experience (QoE)-aware Deep Q-Network (DQN) admission policy on a real OpenAirInterface (OAI) gNodeB (gNB) across three network slices (eMBB, URLLC, mMTC), and compare it against a static baseline, the SLA-only reward of our prior work, and a best-possible static-at-cap policy. After identifying and correcting a calibration issue in our QoE estimation, a fully powered live re-evaluation (128 episodes per arm) shows DQN under the QoE-aware reward sustaining 100% SLA compliance throughout, significantly ahead of the static baseline (Fisher exact $p = 0.0070$, $p = 0.021$ after Holm-Bonferroni correction). The SLA-only reward and the static-at-cap arm converge to a statistically indistinguishable rate (126 of 128 episodes each) that remains only numerically, not significantly, above the static baseline's 120 of 128: the collapse-avoidance edge our earlier, smaller-sample evaluation suggested for the SLA-only reward does not hold at this larger scale. Checking five further independently-trained SLA-reward checkpoints shows live robustness varies substantially by training seed despite equivalent offline convergence, traced in part to the offline training environment's fidelity to real traffic. We additionally live-evaluate a congested, URLLC-preservation scenario: unlike its offline counterpart, where DQN measurably trades eMBB compliance for URLLC under an imposed resource constraint, every arm is fully SLA-compliant live at this rig's traffic scale, though, at a single-seed, 2-episode pilot, the QoE-aware reward still yields a materially higher URLLC Mean Opinion Score at the same compliance level. We also report the control-loop latency and model footprint needed for practical deployment. These findings demonstrate the feasibility, and the honest limits, of deploying DRL-driven SAC on real Open RAN hardware.

Index Terms—5G, Network Slicing, Slice Admission Control, Deep Reinforcement Learning, Quality of Experience, Open RAN

I. INTRODUCTION AND BACKGROUND

Network slicing (NS) lets a single physical 5G deployment serve heterogeneous service classes under per-slice Service Level Agreements (SLAs): Ultra-Reliable Low-Latency Communication (URLLC) requires sub-millisecond latency and high reliability, Enhanced Mobile Broadband (eMBB) demands high throughput, and Massive Machine-Type Communication (mMTC) accommodates large numbers of low-power devices. Slice Admission Control (SAC) decides whether an

incoming slice request can be accommodated given current resource availability. Static, threshold-based SAC cannot adapt its allocation as demand shifts, motivating context-aware, learning-based approaches.

Our prior work applied DRL to SAC under gNodeB (gNB) congestion, using SLA-driven reward shaping to prioritise URLLC traffic, and later extended this to a third slice (mMTC) with a load-balancing objective across multiple gNBs [1], [2], both validated only in simulation. Our systematic review of the 2024–2026 literature organises this space into three evidence tiers – literature-supported, our own preliminary simulated studies, and a proposed, not-yet-validated graph-attention multi-agent federated-learning (GAT-MARL-FL) framework – and identifies validated deployment as one of the field's persistent gaps [3]. This paper closes that gap for our own formulation: we realise the same admission-control problem on a real OAI gNB over its native E2 control interface, with the radio channel still simulated (rfsim), and centre the evaluation on QoE rather than SLA compliance alone.

This paper makes the following contributions:

- 1) We deploy a DRL-based SAC policy on a real O-RAN E2 control loop and identify a structural difference from our earlier simulated work: only a per-slice resource ceiling is exposed, not a per-flow accept/reject primitive, so admission is realised as a ceiling adjustment.
- 2) We compare a QoE-aware reward against the SLA-only reward of [1], [2] and a static-at-cap arm on the same live hardware, isolating genuine admission timing from simply a more generous fixed allocation.
- 3) We report the control-loop latency and model size a real deployment needs, a QoE-calibration bug found and fixed, and the fully powered re-evaluation it enabled.
- 4) We evaluate a congested, multi-slice scenario offline and, at a smaller scale, live, showing where the offline scarcity mechanism does and does not carry over to real hardware.
- 5) We show live robustness varies substantially across independently-trained checkpoints despite equivalent offline convergence, identifying a corresponding offline-environment limitation.
- 6) This single-agent, single-gNB deployment validates the

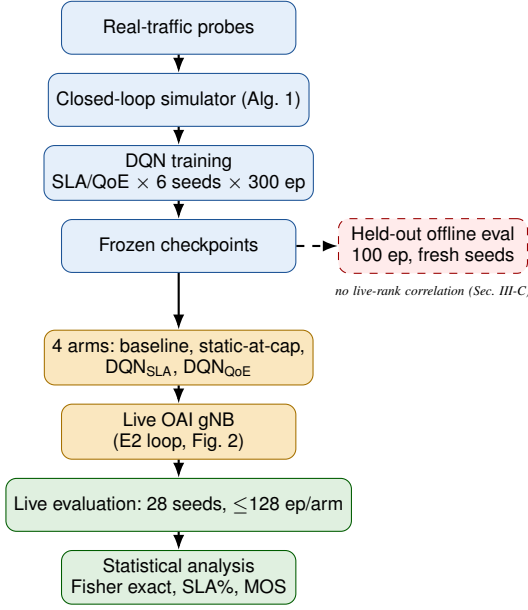


Fig. 1. End-to-end methodology: frozen checkpoints deployed live with no further learning, reduced to Table I/Fig. 4. A held-out offline evaluation runs alongside this pipeline but, per Section III-C, does not predict its outcome.

live control loop our systematic review’s proposed GAT-MARL-FL framework would extend, a deliberate reduced case rather than an omission [3].

The remainder of this paper is organised as follows: Section II reviews related work on DRL-driven SAC. Section III describes the live testbed, the problem formulation, and the reward function. Section IV presents and discusses the results. Section V concludes the paper and outlines future work.

II. LITERATURE REVIEW

TODO(MANOJ): Literature review to be added here, following the style of Section II in our prior conference papers [1], [2] (narrative summary of ~8–10 related DRL-driven SAC / real-testbed studies, each introduced with “The authors in [X] proposed...”), concluding with a short paragraph positioning this paper against that related work. Candidate sources are already screened in our systematic review [3]. Keep this to about half a column given the 4.5-page budget.

III. METHODOLOGY

Fig. 1 summarises the pipeline detailed in this section: offline training against a closed-loop simulator (Section III-C) produces frozen checkpoints, deployed on the live testbed below with no further learning.

A. Testbed

Fig. 2 shows the resulting software architecture.

We deploy a real OAI gNB (ORANSlice fork [4]) with a simulated radio channel (rfsim), an Open5GS core, and three UEs, one per slice. The gNB has a nominal capacity of 106 Physical Resource Blocks (PRBs, $\mu = 1$ numerology). Unlike our earlier simulated work, the real O-RAN E2 interface does not expose a binary accept/reject action; its only control

primitive is a per-slice ratio ceiling on a 0–100 scale, enforced by the MAC scheduler [5]. We therefore realise the admission decision as a ceiling adjustment: an accept decision raises a slice’s ceiling toward a calibrated maximum, and a reject decision lowers it toward a floor. Ceiling values were calibrated by direct measurement on the live rig rather than assumed, since the 0–100 ratio scale does not map linearly onto real PRBs at this cell’s configuration. The static baseline holds its ceiling fixed for the whole episode; a second static arm, *static-at-cap*, instead pins every slice’s ceiling at its calibrated maximum from the start, isolating whether a more generous fixed allocation alone can match a learned policy. Learned arms are DQN policies [6], trained offline against a closed-loop simulated traffic source, then evaluated live with frozen weights, over an E2 agent/xApp loop that all four arms issue admission decisions through.

B. Problem Formulation

The gNB state at time t is defined as

$$s_t = \left(\frac{U_s(t)}{B}, C_t, \frac{L_s(t)}{L_{\max}} \forall s \in \mathcal{S} \right) \quad (1)$$

where $U_s(t)$ is the resource allocation of slice $s \in \mathcal{S} = \{\text{eMBB, URLLC, mMTC}\}$, B is the total resource budget on the ceiling’s normalised 0–100 scale, C_t is the current congestion level, and $L_s(t)$ is slice s ’s queue length, normalised by its maximum capacity $L_{\max}$. The agent’s action at each step is the ceiling adjustment described above. The reward function is

$$r(s_t, a_t) = \sum_{s \in \mathcal{S}} (\omega_s R_s - \lambda_s) - \mu C_t \|a_t\|_1 + \eta \text{MOS}_t \quad (2)$$

where ω_s and λ_s are slice s ’s reward weight and SLA-violation penalty, R_s is the reward for serving slice s , and $\mu C_t \|a_t\|_1$ penalises total accepted volume by the current congestion level, matching the SLA-driven reward structure of our earlier work [1], [2]. Setting the QoE weight $\eta = 0$ recovers the SLA-only reward used in that prior work; setting $\eta = 1$ adds a Mean Opinion Score (MOS) term – a per-slice closed-form estimator, logarithmic in latency, packet loss, and inverse throughput per Eq. (3) of [3] – giving the QoE-aware reward evaluated here. Both variants are compared on the same live hardware; we also report a priority-weighted compliance score, using each slice’s declared importance weight rather than an unweighted average.

C. Offline Training Environment

A DQN policy needs on the order of hundreds of episodes per seed to converge; across this study’s six independently-seeded checkpoints alone that is 1,800 training episodes, far more live rig time than this paper’s entire live evaluation uses. Training therefore happens offline, against a synthetic, closed-loop traffic source: served traffic is capped by the slice’s own most recently commanded ceiling, and demand the ceiling could not serve persists as backlog rather than vanishing, so an admission decision carries a real, delayed consequence

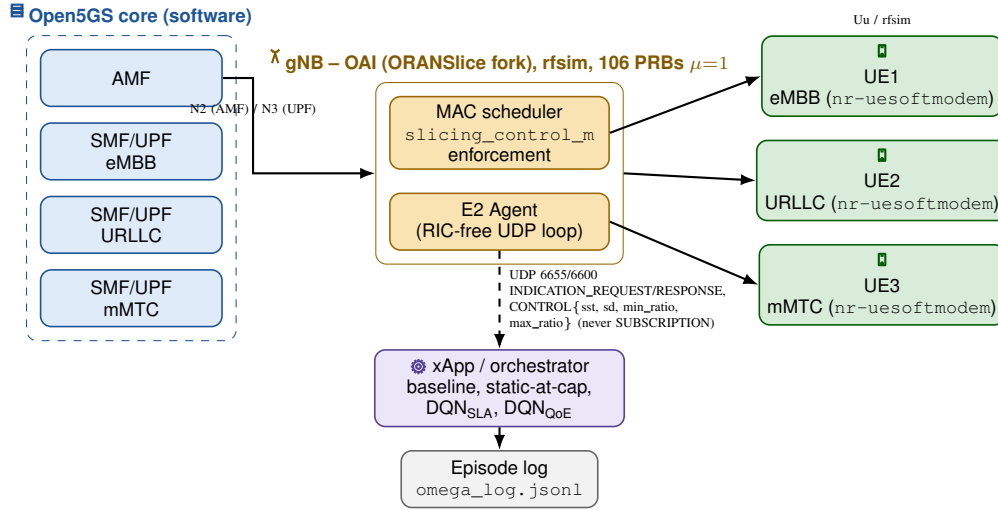


Fig. 2. Testbed software architecture, colour-coded by component: Open5GS core (blue) over N2/N3 to an OAI gNB (yellow) whose MAC scheduler enforces the per-slice ceiling; three `nr-uesoftmodem` UEs (green), one per slice. The RIC-free E2 agent exchanges UDP INDICATION/RESPONSE and CONTROL (`sst, sd, min_ratio, max_ratio`) (never SUBSCRIPTION) messages with the xApp/orchestrator (purple) that all four compared arms run through, which logs every step to `omega_log.jsonl`.

Algorithm 1 Offline closed-loop traffic simulation (one polling step, all gNBs/slices)

Require: per-(gNB,slice) state: offered, backlog, ceiling, UE set

- 1: **for all** gNB g , slice s **do**
- 2: $mean \leftarrow mean_offered_ratio_s \times B \times gnb_load_g$
- 3: $offered_{g,s} \leftarrow \max(0, offered_{g,s} + 0.1(mean - offered_{g,s}) + noise)$
- 4: **if** a reject freed backlog last step (Sec. III-C) **then**
- 5: $backlog_{g,s} \leftarrow \max(0, backlog_{g,s} - relief)$
- 6: **end if**
- 7: $demand \leftarrow offered_{g,s} + backlog_{g,s}$
- 8: $served \leftarrow \min(demand, ceiling_{g,s})$
- 9: $backlog_{g,s} \leftarrow \min(backlog_cap, demand - served)$
- 10: with prob. $churn_prob$, replace one UE's RNTI in (g, s)
- 11: $bler \leftarrow 0.02 + 0.3 \times (backlog_{g,s}/backlog_cap)$
- 12: **for all** UE u in (g, s) 's set, $n \leftarrow |UEs_{g,s}|$ **do**
- 13: emit KPM sample for u : $served/n, bler, backlog_{g,s}/n$
- 14: **end for**
- 15: **end for**
- 16: agent observes the resulting state s_t
- 17: agent sends ceiling $_{g,s} \forall (g, s)$ for the next step

– unlike an earlier, open-loop source we also implemented, under which always accepting was trivially reward-optimal. Algorithm 1 summarises the closed-loop update, one full polling step across every (gNB, slice) pair.

This source's demand mean is tied to values measured live on this rig, but its temporal dynamics are not themselves validated against live traffic. This matters in practice: held-out offline evaluation of the same six checkpoints does not rank-correlate with their live compliance (Section IV-A) – a checkpoint with a perfect live record can look indistinguishable offline from one that fails live, traced in part to a simulator parameter (the backlog capacity) never validated against real SLA-margin magnitude. Offline training itself still produces competent policies, but cannot presently be used to rank or pre-select which checkpoint will – every number in this paper's Results is therefore a live measurement, not an offline projection.

D. Evaluation Setup

Each of the four arms was evaluated live across 28 random seeds, with frozen policy weights, reached in three extensions: an initial 3 seeds at 2 episodes each, then 8 more at the full 5-episode protocol once a QoE-calibration issue (Section IV-B) was corrected (46 episodes/arm), then 17 more at the same protocol for the fully powered 128 episodes/arm reported here – large enough that a tied Fisher exact test is a genuine null result, not merely underpowered. The static-at-cap arm reuses the static baseline's own control code, ceiling parameter changed only in configuration; training followed Section III-C's procedure, 300 episodes per seed.

IV. RESULTS AND DISCUSSION

A. Live Testbed Results

We classify a live episode as *fully compliant* if every per-step SLA margin – the worse of each slice's queue-occupancy and packet-loss margin – stays positive for all three slices across the entire episode, and *collapsed* otherwise. Table I summarises SLA compliance for all four arms across 128 live episodes each, following a fully powered re-evaluation under the corrected QoE calibration described in Section IV-B. DQN under the QoE-aware reward sustains 100% SLA compliance on every slice, in every episode (128/128 episodes fully compliant); DQN under the SLA-only reward and the static-at-cap arm reach an identical, slightly lower rate (126/128 each); the static baseline, whose commanded ceiling stays flat at its starting value for the whole episode rather than rising toward the calibrated maximum, trails both at 120/128.

Since the paper's central comparison is DQN-QoE against DQN-SLA rather than the static baseline, Fig. 3 plots both directly on the one seed where DQN-SLA collapses: between steps 33–37, DQN-SLA drops URLLC's ceiling to under half its cap while DQN-QoE holds URLLC at its cap throughout –

TABLE I
LIVE SLA COMPLIANCE (%), 128 EPISODES/ARM

Arm	eMBB	URLLC	mMTC
Static baseline	94.0	93.8	93.9
Static-at-cap	99.1	98.4	98.5
DQN (SLA reward)	98.8	98.4	98.5
DQN (QoE reward)	100.0	100.0	100.0

Commanded PRB ceiling: DQN-SLA vs. DQN-QoE, seed 955 ep.\ 1

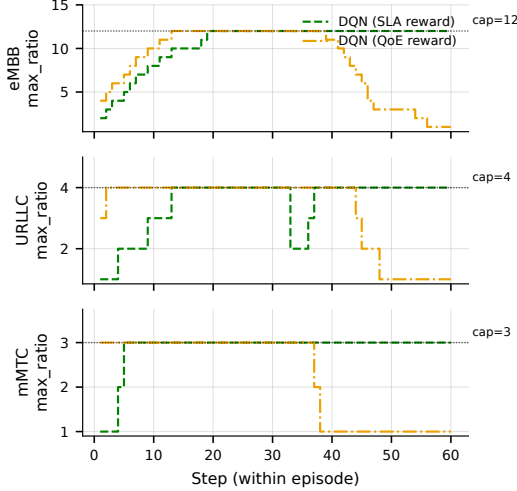


Fig. 3. Commanded ceiling per slice, DQN-SLA vs. DQN-QoE, same seed (955) and episode: the one where DQN-SLA collapses (URLLC 0% compliant steps) while DQN-QoE, on the identical seed/episode, stays fully compliant.

the mechanism behind that episode’s 0% vs. 100% URLLC-compliant-step outcome.

A natural question is whether DQN’s advantage simply reduces to a more generous fixed ceiling. Our earlier, smaller-sample evaluation suggested static-at-cap collapses meaningfully more often than DQN-SLA (27% vs. 0%, Fisher exact $p = 0.0149$); at this paper’s fully powered, nearly $3\times$ larger 128-episode/arm sample, that edge does not hold – DQN-SLA and static-at-cap reach an identical 126/128 (Fisher exact $p = 1.0$), a genuine, now-repeated null result. Both separate further from the baseline’s 120/128 than at 46 episodes/arm, but only DQN-QoE’s gap is significant: raw $p = 0.10$ (DQN-SLA, static-at-cap) and $p = 0.0070$ (DQN-QoE) against baseline, adjusting to $p = 0.21$ and $p = 0.021$ after Holm-Bonferroni correction across the three-arm family. The QoE-aware reward’s advantage survives that correction; the SLA-only reward’s does not.

Fig. 4 shows this episode-by-episode: DQN under the QoE-aware reward sits at exactly 100% in every episode, while the other three arms each show the same bimodal split between fully-compliant and collapsed episodes.

A checkpoint is a single set of frozen DQN weights deployed live with no further learning. Since only one DQN-SLA checkpoint was evaluated above, we trained 5 more and live-evaluated all six at a reduced 21-episode protocol (the full 128-episode protocol on all six would cost roughly six times this paper’s live-evaluation budget). Despite indistinguishable

offline convergence, live records varied substantially (3/5 perfect, one matching the original 19/21, one collapsing to 13/21, worse than baseline): live robustness is not predictable from offline convergence alone, the practical face of the offline-environment limitation noted in Section III-C.

B. Deployment Feasibility

Measured directly against the real gNB, the control loop’s E2 message round trip has a median latency of 0.57 ms, and the DQN model itself (under 20,000 parameters) selects an action in under $70\mu\text{s}$ on CPU. Both are a small fraction of the 5-second control interval used in this study, indicating the learned policy adds negligible overhead to the control loop.

Comparing offline predictions against live results showed a good match for eMBB but not URLLC/mMTC, trained under a different demand assumption than observed live. This also surfaced a QoE-calibration bug: the MOS estimator’s throughput term used a slice’s PRB-usage ratio at run time, but eMBB’s coefficients had been fit assuming an absolute per-UE PRB count – a real ratio of 0.15 was read as almost-starved, pinning eMBB’s MOS at the scale floor regardless of policy quality. Refitting against the ratio actually used lifted eMBB’s MOS at real operating points from a uniform 1.00 to 2.29/3.62/4.11; Table I reports the resulting re-evaluation.

C. Congested Scenario (Offline)

Sections IV-A/B use one constant demand level per episode; to test a harder scenario, we evaluate the same policies offline under continuously varying, randomised multi-slice traffic with genuine cross-slice contention (combined demand exceeding the shared PRB pool scales down every slice’s served allocation proportionally). Pooled over 11 seeds (165 held-out episodes/arm), both DQN variants raise URLLC compliance above the static baseline’s 19.7% (to 27.7%/25.0% for SLA-only/QoE-aware) by trading off eMBB (32.7%→7.7%/8.2%) and mMTC (19.8%→9.4%/11.6%) – the same URLLC-first behaviour our earlier simulated work targeted [1], [2], now under real cross-slice contention, replicating on an independent 8-seed extension. Weighted by declared priority, the static baseline still scores higher overall (24.9% vs. 19.1%/17.9% for DQN), since eMBB’s importance outweighs URLLC’s gain.

We additionally live-evaluated the same checkpoints at a smaller pilot scale (one seed, 2 episodes/arm): every arm, including the static baseline, was fully SLA-compliant live – a scale mismatch, not a contradiction, since this rig’s calibrated ceilings never approach the physical contention the offline budget forces. The QoE-aware reward nonetheless produced a materially higher URLLC MOS (4.83) than SLA-only (2.44) or the static baseline (2.66) at the same 100% compliance (pilot: 1 seed, 2 ep/arm).

V. CONCLUSION AND FUTURE WORK

This paper moved our prior simulated DRL-based slice admission control work [1], [2] onto a real OAI gNB over its native E2 control loop, radio channel still simulated. After

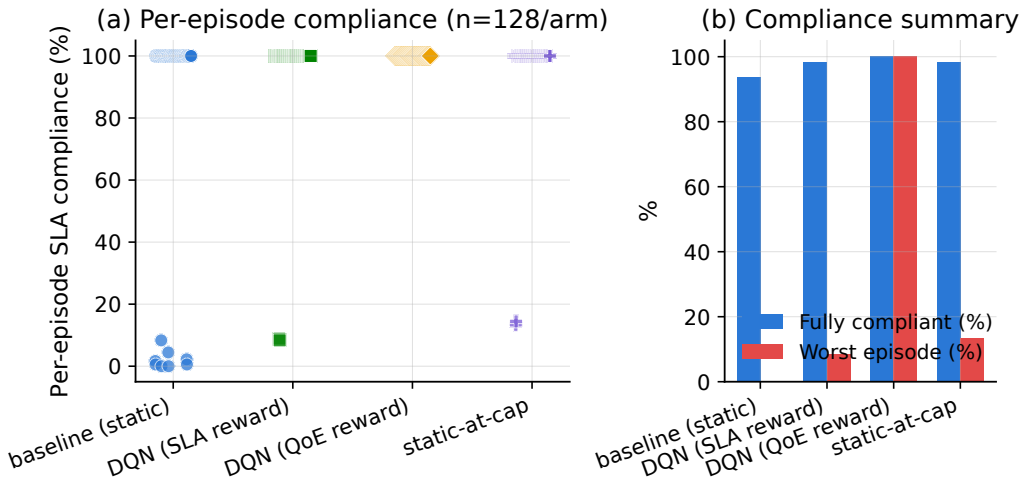


Fig. 4. Per-episode SLA compliance (%). (a) One point per episode. (b) Fraction of episodes fully SLA-compliant and worst single-episode compliance. Baseline, static-at-cap, and DQN under the SLA-only reward all show the same bimodal split; only DQN under the QoE-aware reward stays uniformly compliant.

correcting a QoE-calibration issue, a fully powered live re-evaluation (128 episodes/arm) showed the QoE-aware reward sustaining 100% SLA compliance, now significantly ahead of the static baseline; the SLA-only reward and static-at-cap remained statistically indistinguishable from each other and only numerically ahead of the baseline, revising our earlier, smaller-sample finding of a significant SLA-only edge. Control-loop latency and model overhead were negligible. An offline congested-traffic evaluation showed the learned policy can prioritise URLLC, though not yet as a clear win once priorities are weighted; live, no arm was forced into that tradeoff, though the QoE-aware reward still gave URLLC a materially better experience score.

Our future work will improve the offline training environment’s fidelity to real traffic (Section III-C) and extend this single-agent, single-gNB deployment toward the topology-aware, multi-agent, federated GAT-MARL-FL framework proposed in our systematic review [3]. This paper substantially advances two of that review’s five validation directions – calibrating QoE against real operating points, and O-RAN execution on a validated live testbed; the remaining topology-scalability, FL/privacy, and disruption-resilience directions are our planned next paper (#5), building directly on the live control loop validated here.

ACKNOWLEDGMENT

This research was supported by the Ministry of Higher Education (MOHE), Malaysia through the Fundamental Research Grant Scheme (FRGS/1/2024/ICT06/TAYLOR/02/1).

REFERENCES

[1] M. P. Kunasegran, W. L. Pang, and S. K. Phang, “Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion,” in *2025 Multimedia University Engineering Conference (MECON)*, 2025, pp. 1–6.

[2] —, “SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN,” in *2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, 2025, pp. 207–212.

[3] —, “Comprehensive review of optimisation techniques for user-centric distributed network slicing in 5G networks,” *IEEE Access*, 2026, vERIFY: final volume/issue/DOI once formally published – title page seen locally still shows placeholder publication-date/DOI fields.

[4] H. Cheng, S. D’Oro, R. Gangula, S. Velumani, D. Villa, L. Bonati, M. Polese, G. Arrobo, C. Maciocco, and T. Melodia, “ORANSlice: An open-source 5G network slicing platform for O-RAN,” in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom)*, 2024, repository: <https://github.com/wineslab/ORANSlice>. This work’s fork: OAI 2024.w28 base, commit b9bcc9b – citation confirmed via the repository’s own “how to cite” guidance, 2026-07-30.

[5] O-RAN Alliance, “O-RAN working group 3, near-RT RIC architecture and E2 general aspects and principles (E2AP),” O-RAN Alliance, Tech. Rep., 2023, vERIFY: exact spec number/version (e.g. O-RAN.WG3.E2AP-v03.00) and year.

[6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.